NAQLA

المختبر 2

المختبر 2 – واجهة الحجز البرمجية

هندسة الواجهات الخلفية بالوكلاء الذكيين

هندسة الواجهات الخلفية بالوكلاء الذكيين – بناء أنظمة خلفية قابلة للتوسع بالذكاء الاصطناعي ومن أجله

المختبر 2 — واجهة الحجز البرمجية

نظرة عامة

الهدف

بنيتَ بالأمس نموذج بيانات «شحنة». واليوم تضع أمامه واجهة برمجة (API). بحلول الساعة 15:45 سيتمكن العميل من التسجيل، وتسجيل الدخول، وعرض قائمة المدن، والحصول على عرض سعر (Quote)، وحجز شحنة (Shipment) انطلاقاً من ذلك العرض، وعرض شحناته الخاصة وتفاصيل كل منها، وإلغاء إحداها، ومشاهدة خطها الزمني للتتبّع (Tracking). أما الموظفون (المرسِلون (Dispatchers) والمديرون) فيستطيعون عرض أي شحنة وإلغاءها. ولا أحد غيرهم.

عادة اليوم هي **الاختبار أولاً مع الوكيل** (Test-First with an Agent): يكتب الوكيل الاختبار الفاشل، وتقرؤه أنت، *ثم* يكتب الوكيل الشيفرة. الاختبار الذي قرأته مواصفةٌ توافق عليها. والشيفرة بلا اختبار ادّعاءٌ لم تتحقق منه.

ما ستبنيه

الجزء	مكانه
`PricingService` مع اختبارات الوحدات	`app/Services/PricingService.php`، `tests/Unit/PricingServiceTest.php`
التسجيل وتسجيل الدخول (رموز Sanctum)	`app/Http/Controllers/Api/AuthController.php`
`GET /api/cities`	`app/Http/Controllers/Api/CityController.php`
`POST /api/quotes` مع `StoreQuoteRequest`	`app/Http/Requests/StoreQuoteRequest.php`
الشحنات: الإنشاء، والعرض كقائمة، وعرض التفاصيل، والإلغاء، والتتبّع	`app/Http/Controllers/Api/ShipmentController.php`، `app/Services/BookingService.php`
`ShipmentPolicy` (المالك أو الموظفون)	`app/Policies/ShipmentPolicy.php`
`QuoteResource`، `ShipmentResource`، `ShipmentEventResource`	`app/Http/Resources/`
20 اختباراً ناجحاً على الأقل	`tests/Unit/`، `tests/Feature/`

نقطة البداية

```
git checkout day1-end
git switch -c my-day2
sail up -d
sail artisan migrate:fresh --seed
sail artisan test
```

يجب أن تنجح جميع اختبارات اليوم الأول قبل أن تبدأ (يحتوي المرجع `day1-end` على **23** اختباراً). إن لم تنجح، فأخبر مدرّبك الآن، لا عند الساعة 15:00.

الميزانية الزمنية

الجزء	الخطوة	الدقائق
الجزء الأول (11:00-12:30)	1. اختبارات PricingService، يدوياً	20
	2. تنفيذ PricingService	15
	3. التسجيل وتسجيل الدخول	20
	4. المدن وعروض الأسعار	25
	نقطة التحقق 1	10
الجزء الثاني (14:15-15:45)	5. حجز شحنة	25
	6. القائمة، وعرض التفاصيل، والسياسة	20
	7. الإلغاء والتتبّع	20
	8. قائمة تحقق مراجعة الشيفرة بالذكاء الاصطناعي	15
	نقطة التحقق 2 والإيداع	10

أعراف الاستجابة (اقرأها مرة واحدة، واستخدمها طوال اليوم)

تستخدم كل مختبرات هذا الأسبوع أشكال JSON نفسها. ضع هذا الجدول بجانب محررك.

الحالة	رمز الحالة	جسم الاستجابة
التسجيل	201	`{"token": "...", "user": {...}}`
تسجيل الدخول	200	`{"token": "...", "user": {...}}`

الحالة	رمز الحالة	جسم الاستجابة
مورد واحد	201 / 200	`{"data": {...}}`
قائمة مقسّمة إلى صفحات	200	`{"data": [...], "links": {...}, "meta": {...}}`
خطأ في التحقق	422	`{"message": "...", "errors": {"field": ["..."]}}`
رمز مفقود أو غير صالح	401	`{"message": "Unauthenticated."}`
غير مسموح	403	`{"message": "This action is unauthorized."}`
غير موجود	404	`{"message": "..."}`

المبالغ المالية (`price_lyd`) تكون دائماً **سلسلة نصية** (String) بخانتين عشريتين، مثل `"322.00"` . ولا تكون أبداً عدداً عشرياً عائماً (Float) في JSON.

> **نصيحة**
>
> أرسل دائماً `-H "Accept: application/json"` مع curl. فبدونه يعامل Laravel فشل التحقق أو غياب الرمز على أنه طلب من متصفح، فيجيب بإعادة توجيه (Redirect) بدلاً من JSON.

الجزء الأول – التسعير، والمصادقة، وعروض الأسعار (11:00–12:30)

الخطوة 1 – اكتب اختبارات PricingService يدوياً (20 دقيقة)

ماذا ولماذا. التسعير مال. إن أخطأ فيه الوكيل، فإما أن يدفع العميل أكثر مما يجب، أو تخسر «شحنة» مالاً في كل رحلة. القاعدة قصيرة بما يكفي لتكتب *أنت* اختباراتها، وعليك أن تفعل: فالاختبارات هي المواصفة.

> **تحذير**
>
> **يدوياً.** اكتب ملف الاختبار هذا بنفسك. لا تطلب من الوكيل كتابته ولا «اقتراح حالات».

القاعدة (من `docs/requirements.md`):

```
price = 50 LYD base + km × rate_per_km[vehicle_type] + weight_kg × 0.02
rate_per_km: van 0.8, box_truck 1.2, flatbed 1.5, reefer 1.9
Round to 2 decimals. Same city = 0 km.
```

للخدمة دالتان عامتان:

- `price(int $originCityId, int $destinationCityId, VehicleType $vehicleType, int $weightKg): string` – تبحث عن المسافة في `city_distances` (0 كم للمدينة نفسها) وتُرجع السعر سلسلةً نصية، مثل `"322.00"`.
- `createQuote(User $user, int $originCityId, int $destinationCityId, VehicleType $vehicleType, int $weightKg): Quote` – تحفظ عرض سعر بذلك السعر، صالحاً لمدة 15 دقيقة.

تأتي المسافات من `CitySeeder` الذي كتبته في اليوم الأول، لذا يُقلع الاختبار Laravel ويملأ جدول المدن بالبذّارة (Seeder). أضف أولاً هذه الدالة المساعدة إلى نهاية `tests/Pest.php`:

```php
function city(string $code): App\Models\City
{
    return App\Models\City::where('code', $code)->firstOrFail();
}
```

احسب القيم المتوقعة على الورق أولاً، ثم أنشئ `tests/Unit/PricingServiceTest.php`:

```php
<?php

use App\Enums\VehicleType;
use App\Services\PricingService;
use Database\Seeders\CitySeeder;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

// Pricing reads distances from the database, so these tests boot Laravel.
uses(TestCase::class, RefreshDatabase::class);

beforeEach(function () {
    $this->seed(CitySeeder::class);
    $this->pricing = new PricingService;
});

it('prices Tripoli to Misrata by box truck', function () {
    // 50 + 210 km × 1.2 + 1000 kg × 0.02 = 322
    expect($this->pricing->price(city('TIP')->id, city('MRA')->id, VehicleType::BoxTruck,
        1000))->toBe('322.00');
});

it('uses the rate for each vehicle type', function (VehicleType $type, string $expected) {
    // Tripoli → Zawiya is 45 km, 500 kg adds 10 LYD
    expect($this->pricing->price(city('TIP')->id, city('ZAW')->id, $type, 500))
        ->toBe($expected);
})->with([
    [VehicleType::Van, '96.00'],
    [VehicleType::BoxTruck, '114.00'],
    [VehicleType::Flatbed, '127.50'],
    [VehicleType::Reefer, '145.50'],
]);

it('treats the same city as zero km', function () {
    expect($this->pricing->price(city('BEN')->id, city('BEN')->id, VehicleType::Van, 100))
        ->toBe('52.00');
});

it('prices a long trip without a thousands separator', function () {
    // 50 + 1020 km × 1.9 + 2500 kg × 0.02 = 2038
    expect($this->pricing->price(city('TIP')->id, city('BEN')->id, VehicleType::Reefer, 2500))
        ->toBe('2038.00');
});

it('rounds to two decimals', function () {
    // 50 + 45 × 0.8 + 333 × 0.02 = 92.66
    expect($this->pricing->price(city('TIP')->id, city('ZAW')->id, VehicleType::Van, 333))
        ->toBe('92.66');
});
```

تحقّق من حسابك للحالة الأولى: 50 + 210 × 1.2 + 1000 × 0.02 = 50 + 252 + 20 = **322.00**. واحسب البقية بنفسك قبل أن تثق بها. تفترض هذه الأرقام المسافات المبذورة: طرابلس-مصراتة (TIP-MRA) 210 كم، وطرابلس-الزاوية (TIP-ZAW) 45 كم، وطرابلس-بنغازي (TIP-BEN) 1020 كم؛ وإن كانت بذّارتك مختلفة، فاستخدم قيمك.

شغّله. يجب أن يفشل، لأن الصنف غير موجود بعد:

```
sail artisan test --filter=PricingService
```

```
  FAILED  Tests\Unit\PricingServiceTest > it prices a trip with ...
  Error: Class "App\Services\PricingService" not found
```

الخطوة 2 — نفّذ PricingService مع الوكيل (15 دقيقة)

ماذا ولماذا. الآن يكتب الوكيل الشيفرة لتجتاز اختباراتك *أنت*. ولا يحق له تعديلها.

🤖 **اسأل وكيلك الذكي**

طلب للوكيل

خطّط أولاً، ثم انتظر موافقتي.

أنشئ App\Services\PricingService بحيث ينجح tests/Unit/PricingServiceTest.php. لا تعدّل ملف الاختبار ذاك.

- الدالة price(int $originCityId, int $destinationCityId, VehicleType $vehicleType, int $weightKg): string: الأساس 50 + الكيلومترات × المعدّل + الوزن × 0.02، والمعدّلات: van 0.8، box_truck 1.2، flatbed 1.5، reefer 1.9. أرجِع سلسلة نصية بخانتين عشريتين بالضبط.
- الدالة distanceKm(int $originCityId, int $destinationCityId): int: تُرجع 0 عندما يكون المنشأ والوجهة المدينة نفسها، وإلا فقيمة صف CityDistance (باستخدام valueOrFail، بحيث تُرمى استثناءات عند غياب المسافة).
- الدالة createQuote(User $user, int $originCityId, int $destinationCityId, VehicleType $vehicleType, int $weightKg): Quote: تنشئ عرض سعر للمستخدم بذلك السعر، مع expires_at = الآن + 15 دقيقة.

ضع الرسوم والمعدّلات في ثوابت الصنف (BASE_FEE_LYD، RATE_PER_KM_LYD، RATE_PER_KG_LYD، QUOTE_TTL_MINUTES)، لا في ملفات الإعداد (config). شغّل sail artisan test -filter=Pricing واعرض المخرجات.

بالإنجليزية:

```
Plan first, then wait for my approval.

Create App\Services\PricingService so that tests/Unit/PricingServiceTest.php
passes. Do NOT modify that test file.
- price(int $originCityId, int $destinationCityId, VehicleType $vehicleType,
  int $weightKg): string
  base 50 + km * rate + weight * 0.02, rates van 0.8, box_truck 1.2,
  flatbed 1.5, reefer 1.9. Return a string with exactly 2 decimals.
- distanceKm(int $originCityId, int $destinationCityId): int
  0 when origin and destination are the same city, otherwise the
  CityDistance row (valueOrFail, so a missing distance throws).
- createQuote(User $user, int $originCityId, int $destinationCityId,
  VehicleType $vehicleType, int $weightKg): Quote
  creates the user's quote with that price and expires_at = now + 15 minutes.
Put the fee and rates in class constants (BASE_FEE_LYD, RATE_PER_KM_LYD,
RATE_PER_KG_LYD, QUOTE_TTL_MINUTES), not in config.
Run `sail artisan test --filter=Pricing` and show the output.
```

ما الذي تتحقق منه.

- جيد: تُنسَّق النتيجة باستخدام `number_format($total, 2, '.', '')`، الذي يعطي `"322.00"`، لا `322` ولا `"322.0"`.

- جيد: المعدّلات موجودة في مصفوفة ثابتة واحدة مفاتيحها أنواع المركبات. مكان واحد لتغيير الأسعار.
- علامة خطر: يُظهر `git diff tests/Unit/PricingServiceTest.php` أي تغيير. تراجع عنه.
- علامة خطر: `number_format($total, 2)` دون وسيطي الفواصل. فللأسعار من 1,000 دينار فما فوق يُرجع `"2,038.00"`، فتفشل حالة الرحلة الطويلة. جيد: لقد التقطه الاختبار.

```
sail artisan test --filter=Pricing
```

```
   PASS  Tests\Unit\PricingServiceTest
  ✓ it prices Tripoli to Misrata by box truck
  ✓ it uses the rate for each vehicle type with ...
  ...
  ✓ it rounds to two decimals

  Tests:    8 passed
```

أودِع التغييرات (Commit):

```
git add app/Services tests
git commit -m "feat: pricing service"
```

الخطوة 3 — التسجيل وتسجيل الدخول (20 دقيقة)

ماذا ولماذا. يُصدر Sanctum رموز API معتمة (Opaque Tokens). يرسلها العميل على هيئة `Authorization: Bearer <token>`. المصادقة (Authentication) تجيب عن سؤال *من أنت*؛ وسياسات (Policies) الغد تجيب عن سؤال *ما الذي يحق لك فعله*.

🤖 **اسأل وكيلك الذكي**

طلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. الاختبارات أولاً، ثم الشيفرة.

1. اكتب اختبارات ميزات Pest في tests/Feature/AuthTest.php:
   - التسجيل بالاسم والبريد الإلكتروني وكلمة المرور (8 أحرف على الأقل) يُرجع 201 وJSON بالشكل {"token": string, "user": {id, name, email, role}}؛ ويكون دور user.role = "customer" مع المستخدم في قاعدة البيانات customer حتى لو أرسل الطلب role=admin.
   - التسجيل ببريد إلكتروني مكرر يُرجع 422 مع errors.email.
   - تسجيل الدخول ببيانات اعتماد صحيحة يُرجع 200 {"token": string, "user": {...}}.
   - تسجيل الدخول بكلمة مرور خاطئة يُرجع 422 مع errors.email.
   - الطلب GET /api/shipments دون رمز يُرجع 401. شغّلها وأرني أنها تفشل.
2. ثم نفّذ AuthController بدالتي loging register، مستخدماً أصناف FormRequest، Hashg، user-$g->createToken('api')->plainTextToken. الحقل role غير قابل للتعبئة: تضبط register بنفسها $user->role = UserRole::Customer. توجد المتحكمات في App. المسارات في routes/api.php: POST /auth/register، POST /auth/loging. أضف مجموعة مسارات auth:sanctum فارغة لنقاط النهاية التالية. اجعل الفرق صغيراً. لا تُرجع تجزئة كلمة المرور أبداً.

بالإنجليزية:

```
Plan first, then wait for my approval. Tests first, then code.

1. Write Pest feature tests in tests/Feature/AuthTest.php:
   - register with name, email, password (min 8) returns 201
     and JSON {"token": string, "user": {id, name, email, role}} with
     user.role = "customer", and the user's role in the database is customer
     even if the request sends role=admin.
   - register with a duplicate email returns 422 with errors.email.
   - login with correct credentials returns 200 {"token": string, "user": {...}}.
   - login with a wrong password returns 422 with errors.email.
   - GET /api/shipments without a token returns 401.
   Run them and show me they fail.
2. Then implement AuthController with register and login, using FormRequest
   classes, Hash, and $user→createToken('api')→plainTextToken.
   role is not fillable: register sets $user→role = UserRole::Customer itself.
   Controllers live in App\Http\Controllers\Api.
   Routes in routes/api.php: POST /auth/register, POST /auth/login.
   Add an empty auth:sanctum route group for the endpoints that follow.
Small diff. Never return the password hash.
```

ما الذي تتحقق منه.

- جيد: `role` **ليس** ضمن قواعد `RegisterRequest` ولا يؤخذ من المدخلات. فالإسناد الجماعي (Mass Assignment) لـ `role` سيسمح لأي شخص بأن يجعل نفسه `admin`.
- جيد: يستخدم فشل تسجيل الدخول `ValidationException::withMessages(['email' ⇒ ...])`، الذي ينتج شكل 422 القياسي.
- علامة خطر: يقول تسجيل الدخول "user not found" لبريد غير معروف، و"wrong password" لبريد معروف. هذا يخبر المهاجم بالعناوين المسجّلة. يجب أن تعطي الحالتان الرسالة نفسها.
- علامة خطر: يثبّت الوكيل Sanctum أو ينشر ملف إعداده مرة أخرى. فهو مثبّت مسبقاً في `day1-start`.

جرّبه يدوياً:

```
curl -s -X POST http://localhost/api/auth/register \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"name":"Amal","email":"amal@example.ly","password":"secret-pass-1"}'
```

```
{"token":"1|q8V3cX...","user":
        {"id":4,"name":"Amal","email":"amal@example.ly","role":"customer"}}
```

قيمة `role` هي `customer` لأن `register` يضبطها على الخادم. وهي غير قابلة للتعبئة (Fillable)، لذا يُتجاهل أي حقل `role` في جسم الطلب.

احفظ الرمز في متغير صدفة (Shell Variable) لبقية المختبر:

```
TOKEN=$(curl -s -X POST http://localhost/api/auth/login \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"email":"amal@example.ly","password":"secret-pass-1"}' | jq -r .token)
echo $TOKEN
```

إن لم يكن لديك `jq`، فانسخ الرمز يدوياً: `TOKEN=2|abc...`.

الخطوة 4 – المدن وعروض الأسعار (25 دقيقة)

ماذا ولماذا. `POST /api/quotes` هي أول نقطة نهاية (Endpoint) تستقبل مدخلات حقيقية من المستخدم. يحدث التحقق من المدخلات (Validation) عند الحدود، في `FormRequest`، قبل أن تعمل أي شيفرة من شيفرتك. عرض السعر صالح لمدة 15 دقيقة.

🤖 اسأل وكيلك الذكي

طلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. الاختبارات أولاً، ثم الشيفرة، وفروق صغيرة.

1. GET /api/cities (عامة، دون رمز): تُرجع {"data": [...]} تتضمن id code name_en name_ar لجميع المدن الثماني، مرتبة حسب name_en. استخدم CityResource.
2. POST /api/quotes (auth:sanctum)، والجسم: {origin_city_id, destination_city_id, weight_kg, vehicle_type}
   - StoreQuoteRequest: معرّفا المدينتين مطلوبان exists:cities,id؛ weight_kg مطلوب، عدد صحيح، min:1 max:25000؛ vehicle_type مطلوب (أكبر شاحنة) Rule::enum(VehicleType::class).
   - يأتي عرض السعر من PricingService::createQuote()، لا من الطلب أبداً؛ user_id هو المستخدم المصادَق عليه؛ expires_at = الآن + 15 دقيقة.
   - أرجِع 201 مع QuoteResource: id، والمنشأ والوجهة بصيغة CityResource {id, code, name_en, name_ar}، vehicle_type، weight_kg، price_lyd سلسلةً نصية بخانتين عشريتين، expires_at بصيغة ISO 8601. اختبارات الميزات في tests/Feature/QuoteTest.php: المسار السعيد مع التحقق من السعر، ومدينة غير معروفة ← 422، vehicle_type غير صالح ← 422، والوزن 0 ← 422، وتجاهل price_lyd المرسَل في الجسم، وغير مصادَق عليه ← 401. استخدم Carbon::setTestNow() / $this->freezeTime مع expires_at.

بالإنجليزية:

```
Plan first, then wait for my approval. Tests first, then code, small diffs.

1. GET /api/cities (public, no token): returns {"data": [...]} with id, code,
   name_en, name_ar for all 8 cities, ordered by name_en. Use a CityResource.
2. POST /api/quotes (auth:sanctum), body:
   {origin_city_id, destination_city_id, weight_kg, vehicle_type}
   - StoreQuoteRequest: both city ids required and exist:cities,id;
     weight_kg required integer min:1 max:25000 (the largest truck);
     vehicle_type required and Rule::enum(VehicleType::class).
   - The quote comes from PricingService::createQuote(), never from the
     request; user_id is the authenticated user; expires_at = now + 15 min.
   - Return 201 with QuoteResource: id, origin and destination as
     CityResource {id, code, name_en, name_ar}, vehicle_type, weight_kg,
     price_lyd as a string with 2 decimals, expires_at ISO 8601.
Feature tests in tests/Feature/QuoteTest.php: happy path with price check,
unknown city → 422, bad vehicle_type → 422, weight 0 → 422,
price_lyd sent in the body is ignored, unauthenticated → 401.
Use Carbon::setTestNow / $this→freezeTime() for expires_at.
```

ما الذي تتحقق منه.

- جيد: `Rule::enum(VehicleType::class)` بدلاً من قائمة `in:van,box_truck,...` مكتوبة يدوياً ستبتعد مع الوقت عن التعداد (Enum).
- جيد: يستخدم المتحكم (Controller) `$request->validated()`، لا `$request->all()` أبداً.
- جيد: يحمّل `QuoteResource` العلاقتين `origin` و`destination` تحميلاً مسبقاً (Eager Loading) أو يستلمهما جاهزتين؛ فلا ينفّذ استعلاماً لكل حقل.
- علامة خطر: `price_lyd` موجود ضمن قواعد FormRequest. سيختار العميل سعره بنفسه.
- علامة خطر: يتحقق الاختبار من `expires_at` باستخدام `now()` دون تجميد الوقت. سيفشل مرة في كل مئة تشغيل.

```
curl -s http://localhost/api/cities \
  -H "Accept: application/json" | jq '.data[0]'
```

```
{"id":2,"code":"BEN","name_en":"Benghazi","name_ar":"بنغازي"}
```

احصل على عرض سعر لشاحنة صندوقية من طرابلس إلى مصراتة. ابحث أولاً عن معرّفيهما في قائمة المدن (مع البذّارة المرجعية يكون TIP هو 1 وMRA هو 3):

```
curl -s -X POST http://localhost/api/quotes \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"origin_city_id":1,"destination_city_id":3,"weight_kg":1000,"vehicle_type":"box_truck"}'
```

```
{
  "data": {
    "id": 1,
    "origin": {"id": 1, "code": "TIP", "name_en": "Tripoli", "name_ar": "طرابلس"},
    "destination": {"id": 3, "code": "MRA", "name_en": "Misrata", "name_ar": "مصراتة"},
    "vehicle_type": "box_truck",
    "weight_kg": 1000,
    "price_lyd": "322.00",
    "expires_at": "2026-10-21T11:52:10.000000Z"
  }
}
```

(يعرض curl الأسماء العربية على شكل تسلسلات هروب `\u`؛ أما `jq` فيطبعها بالعربية.) لن تكون قيمة `price_lyd` لديك `"322.00"` إلا إذا كانت المسافة المبذورة بين TIP وMRA هي 210 كم. وإلا فاحسب ما يجب أن تكون عليه وتحقّق.

والآن طلب خاطئ:

```
curl -s -X POST http://localhost/api/quotes \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"origin_city_id":1,"destination_city_id":999,"weight_kg":0,"vehicle_type":"rocket"}'
```

```
{
  "message": "The selected destination city id is invalid. (and 2 more errors)",
  "errors": {
    "destination_city_id": ["The selected destination city id is invalid."],
    "vehicle_type": ["The selected vehicle type is invalid."],
    "weight_kg": ["The weight kg field must be at least 1."]
  }
}
```

نقطة تحقق

نقطة التحقق 1 (12:30)

```
sail artisan test
```

```
  Tests:    ~38 passed
```

سيختلف عددك (ينهي المرجع الكامل `day2-end` اليوم بـ56 اختباراً)؛ المهم هو **صفر إخفاقات**.

التسجيل يُرجع 201 مع رمز؛ وتسجيل الدخول يُرجع 200 مع رمز.

`GET /api/cities` يعمل دون رمز؛ و`GET /api/shipments` دون رمز يُرجع 401 `{"message":"Unauthenticated."}`.

`POST /api/quotes` يُرجع 201 مع `price_lyd` سلسلةً نصية.

```
git add -A
git commit -m "feat: auth, cities and quotes endpoints"
```

الجزء الثاني – الشحنات، والسياسات، والمراجعة (14:15–15:45)

الخطوة 5 – احجز شحنة (25 دقيقة)

ماذا ولماذا. يحوّل الحجزُ عرضَ السعر إلى شحنة. وهنا تعيش قواعد العمل (Business Rules): يمكن استخدام عرض السعر مرة واحدة، ومن مالكه فقط، وقبل انتهاء صلاحيته فقط. يتحقق التطبيق من هذه القواعد ويُرجع أخطاء 422 واضحة، مع قفل صف عرض السعر (`lockForUpdate()`) داخل المعاملة (Transaction)؛ أما

فهرس التفرّد (Unique Index) على `shipments.quote_id` في قاعدة البيانات (من اليوم الأول) فهو خط الدفاع الأخير. وغداً ستضيف مفاتيح عدم التكرار (Idempotency Keys) لتصبح إعادة إرسال الطلب آمنة أيضاً.

🤖 **اسأل وكيلك الذكي**

طلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. الاختبارات أولاً: اكتبها، وشغّلها، وأرني أنها تفشل، ثم نفّذ. فروق صغيرة.

StoreShipmentRequest: - .{quote_id, pickup_at} والجسم ،(auth:sanctum) POST /api/shipments
quote_id مطلوب، عدد صحيح - pickup_at مطلوب، تاريخ، after:now ضع المنطق في App::book(User *Shipment* : (*pickupAt* CarbonInterface $quoteId, int $user. *داخل* : : *DB*
transaction،*حمّلعرضالسعرباستخدام*-()lockForUpdate->(user)*whereBelongsTo* : : *Quote*
(find($quoteId)>. قواعد العمل (ارمِ ValidationException مع errors.quote_id عند مخالفتها): - عرض السعر يخص المستخدم المصادَق عليه (وإلا فـ422، دون الكشف عن أن عرض السعر موجود لدى شخص آخر) - عرض السعر لم تنتهِ صلاحيته (expires_at > now) - عرض السعر لم تستخدمه شحنة من قبل عند النجاح، داخل المعاملة نفسها: - أنشئ Shipment بنسخ المنشأ والوجهة weight_kgg vehicle_typeg price_lydg من عرض السعر (لا من الطلب أبداً)، بالحالة pending، ومرجعٍ يولّده النموذج - أنشئ ShipmentEvent بالحالة pending والملاحظة "Booking received." - أضف
(ShipmentStatus $next, ?string $note)Shipment::transitionTo التي تتحقق عبر canTransitionTo (ValidationException على errors.status إن لم يكن الانتقال مسموحاً)، وتحدّث الحالة، وتكتب ShipmentEvent؛ واستخدمها لنقل الشحنة الجديدة إلى confirmed مع الملاحظة "Booking confirmed." أرجِع 201 مع ShipmentResource: reference، statusg، origing، destinationg، vehicle_typeg، vehicleg (null حتى يُسنَد)، weight_kgg، price_lydg (سلسلة نصية)، pickup_atg، created_atg. الاختبارات في tests/Feature/ShipmentTest.php: المسار السعيد، وعرض سعر منتهي الصلاحية، وعرض سعر مستخدم مسبقاً، وعرض سعر لمستخدم آخر، pickup_atg في الماضي، واختبار يثبت أن فهرس التفرّد في قاعدة البيانات يرفض شحنة ثانية بـquote_id نفسه (أدخلها مباشرة، وتوقّع QueryException).

بالإنجليزية:

```
Plan first, then wait for my approval. Tests first: write them, run them,
show me they fail, then implement. Small diffs.

POST /api/shipments (auth:sanctum), body {quote_id, pickup_at}.
StoreShipmentRequest:
- quote_id required, integer
- pickup_at required, date, after:now
Put the logic in App\Services\BookingService::book(User $user, int $quoteId,
CarbonInterface $pickupAt): Shipment. Inside DB::transaction, load the quote
with Quote::whereBelongsTo($user)->lockForUpdate()->find($quoteId).
Business rules (throw ValidationException with errors.quote_id when violated):
- the quote belongs to the authenticated user (otherwise 422, do not
  reveal that the quote exists for someone else)
- the quote is not expired (expires_at > now)
- the quote has not already been used by a shipment
On success, inside the same transaction:
- create the Shipment copying origin, destination, weight_kg,
  vehicle_type and price_lyd from the quote (never from the request),
  status pending, reference generated by the model
- create a ShipmentEvent with status pending, note "Booking received."
- add Shipment::transitionTo(ShipmentStatus $next, ?string $note) that checks
  canTransitionTo (ValidationException on errors.status if not allowed),
  updates the status and writes a ShipmentEvent; use it to move the new
  shipment to confirmed with note "Booking confirmed."
Return 201 with ShipmentResource: reference, status, origin, destination,
vehicle_type, vehicle (null until assigned), weight_kg, price_lyd (string),
pickup_at, created_at.
Tests in tests/Feature/ShipmentTest.php: happy path, expired quote,
already-used quote, another user's quote, pickup_at in the past,
and a test proving the DB unique index rejects a second shipment with the
same quote_id (insert directly, expect QueryException).
```

ما الذي تتحقق منه.

- جيد: يُنسخ السعر من عرض السعر. اقرأ المتحكم: إن جاء `price_lyd` من `$request` في أي موضع، فهذا خطأ حرج.
- جيد: التحقق من «الاستخدام المسبق» هو `Shipment::where('quote_id', ...)->exists()` أو علاقة `hasOne` على `Quote`، إضافة إلى فهرس التفرّد في قاعدة البيانات.
- جيد: يستخدم اختبار عرض السعر المنتهي `$this->travel(16)->minutes()` بدلاً من الانتظار الفعلي.
- علامة خطر: حالة عرض سعر مستخدم آخر تُرجع 403 أو 404 مع "quote belongs to another user". هذا يسرّب معلومات. عاملها تماماً مثل عرض سعر غير موجود.
- علامة خطر: تُنشأ الشحنة وحدثها الأول خارج معاملة. فإن فشل إدخال الحدث، صارت لديك شحنة بلا سجل.

احجز عرض السعر الذي أنشأته في الخطوة 4:

```
curl -s -X POST http://localhost/api/shipments \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"quote_id":1,"pickup_at":"2026-10-25T08:00:00Z"}'
```

```
{
  "data": {
    "reference": "SHN-7KQ2ZD",
    "status": "confirmed",
    "origin": {"id": 1, "code": "TIP", "name_en": "Tripoli", "name_ar": "طرابلس"},
    "destination": {"id": 3, "code": "MRA", "name_en": "Misrata", "name_ar": "مصراتة"},
    "vehicle_type": "box_truck",
    "vehicle": null,
    "weight_kg": 1000,
    "price_lyd": "322.00",
    "pickup_at": "2026-10-25T08:00:00.000000Z",
    "created_at": "2026-10-21T11:40:02.000000Z"
  }
}
```

إن حصلت على `"This quote has expired. Please request a new quote."`، فقد مرّ أكثر من 15 دقيقة منذ الخطوة 4. أنشئ عرض سعر جديداً. هذه هي القاعدة تعمل.

أرسل الطلب نفسه مرة أخرى. يجب أن تحصل على 422، لا على شحنة ثانية:

```
{"message":"This quote has already been booked.","errors":{"quote_id":["This quote has already
        been booked."]}}
```

احفظ المرجع: `REF=SHN-7KQ2ZD`.

الخطوة 6 — القائمة، وعرض التفاصيل، والسياسة (20 دقيقة)

ماذا ولماذا. يرى العميل شحناته فقط. ويستطيع الموظفون (المرسِل أو المدير) رؤية أي منها. هذه القاعدة مكانها موضع **واحد**، هو السياسة (Policy)، لا أن تتناثر في المتحكمات على شكل عبارات `if`. لا يتضمن `Controller` الأساسي في Laravel 12 السمة (Trait) `AuthorizesRequests`، لذا تستدعي `Gate::authorize('view', $shipment)`.

تستخدم الروابط العامة **مرجع** الشحنة (Reference)، لا المعرّف الرقمي أبداً. وربط النموذج بالمسار (Route Model Binding) يفعل ذلك نيابةً عنك عبر `{shipment:reference}`.

أولاً، اكتب اختبار سياسة واحداً **يدوياً**.

> **تحذير**
>
> **يدوياً.** اكتب الاختبار أدناه بنفسك قبل أن توجّه الطلب إلى الوكيل. إنه قاعدة التفويض (Authorization) في أبسط صورها.

أنشئ `tests/Feature/ShipmentPolicyTest.php`:

```php
<?php

use App\Enums\UserRole;
use App\Models\Shipment;
use App\Models\User;

it('lets the owner view their shipment', function () {
    $shipment = Shipment::factory()->create();

    $this->actingAs($shipment->user, 'sanctum')
        ->getJson("/api/shipments/{$shipment->reference}")
        ->assertOk()
        ->assertJsonPath('data.reference', $shipment->reference);
});

it('forbids another customer', function () {
    $shipment = Shipment::factory()->create();
    $stranger = User::factory()->create(['role' => UserRole::Customer]);

    $this->actingAs($stranger, 'sanctum')
        ->getJson("/api/shipments/{$shipment->reference}")
        ->assertForbidden();
});
```

(استخدم `RefreshDatabase`؛ تحقّق من أن `tests/Pest.php` يطبّقه مسبقاً على مجلد `Feature`؛ وإن لم يكن كذلك، فأضف `uses(Illuminate\Foundation\Testing\RefreshDatabase::class)`.)

شغّله وراقبه يفشل بـ404 (فالمسار غير موجود بعد). والآن سلّم المهمة للوكيل:

🤖 **اسأل وكيلك الذكي**

طلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. لا تعدّل tests/Feature/ShipmentPolicyTest.php؛ اجعله ينجح وأضف إليه.

1. App\Policies\ShipmentPolicy مع view(User, Shipment): true للمالك أو لمستخدم من الموظفين. أضف User::isStaff(): الدور dispatcher أو admin. استخدم التعداد UserRole، لا السلاسل النصية.
2. GET /api/shipments/{shipment:reference} ← Gate::authorize('view', $shipment)، ويُرجع ShipmentResource.
3. GET /api/shipments ← شحنات المستخدم المصادَق عليه فقط، الأحدث أولاً، مقسّمة إلى صفحات بـ15 لكل صفحة، باستخدام ShipmentResource::collection. أضف اختبارات: المرسِل يستطيع عرض أي شحنة؛ ومرجع غير معروف ← 404؛ والقائمة لا تحتوي إلا شحناتي (أنشئ شحنات لمستخدمَين وتحقّق من العدد والمراجع)؛ والقائمة مقسّمة إلى صفحات (meta.per_page = 15).

بالإنجليزية:

```
Plan first, then wait for my approval. Do not modify
tests/Feature/ShipmentPolicyTest.php; make it pass and add to it.

1. App\Policies\ShipmentPolicy with view(User, Shipment): true for the owner
   or a staff user. Add User::isStaff(): role is dispatcher or admin.
   Use the UserRole enum, not strings.
2. GET /api/shipments/{shipment:reference} → Gate::authorize('view', $shipment),
   returns ShipmentResource.
3. GET /api/shipments → the authenticated user's own shipments only,
   newest first, paginated 15 per page, ShipmentResource::collection.
Add tests: dispatcher can view any shipment; unknown reference → 404;
list only contains my shipments (create shipments for 2 users and assert
count and references); list is paginated (meta.per_page = 15).
```

ما الذي تتحقق منه.

- جيد: يبدأ استعلام القائمة من `$request->user()->shipments()`، فلا يمكن أن تظهر صفوف المستخدمين الآخرين أبداً.
- جيد: تقارن السياسة المعرّفات: `$user->id === $shipment->user_id`.
- علامة خطر: تستخدم نقطة نهاية القائمة `Shipment::paginate()` ثم تصفّي في PHP، أو تصفّي بمعامل استعلام `user_id`. كلاهما يسرّب البيانات.
- علامة خطر: تقارن السياسة سلاسل نصية خام (`$user->role === 'dispatcher'`) بدلاً من المرور عبر `isStaff()` والتعداد `UserRole`.

- ملاحظة: ستنفّذ القائمة استعلاماً لكل شحنة من أجل `origin` و`destination` و`vehicle`. هذه هي مشكلة N+1 (N+1 Problem). **اتركها.** ستقيسها وتصلحها غداً.

```
curl -s http://localhost/api/shipments \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" | jq '{count: (.data|length),
        meta: .meta.per_page}'
```

```
{"count": 1, "meta": 15}
```

(لدى أمل شحنة واحدة. ويضيف المرجع `day2-end` أيضاً `DemoShipmentSeeder` الذي يعطي `customer@shahna.test` ثلاثين شحنة مؤكدة، منها المرجع الثابت `SHN-DEMO01`؛ وتستخدمه المختبرات اللاحقة.)

سجّل مستخدماً ثانياً، واحصل على رمزه باسم `TOKEN2`، وحاول قراءة شحنة أمل:

```
curl -s http://localhost/api/shipments/$REF \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN2"
```

```
{"message":"This action is unauthorized."}
```

الخطوة 7 — الإلغاء والتتبّع (20 دقيقة)

ماذا ولماذا. يعيد الإلغاء استخدام القاعدة التي كتبتها واختبرتها بالأمس: `ShipmentStatus::canTransitionTo()`. لا تُعِد تنفيذها في المتحكم أبداً. وكل تغيير في الحالة يكتب أيضاً `ShipmentEvent`، وهو ما تعرضه نقطة نهاية التتبّع.

🤖 اسأل وكيلك الذكي

طلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. الاختبارات أولاً، وفروق صغيرة.

1. أضف cancel(User, Shipment) إلى ShipmentPolicy: المالك أو الموظفون.
2. POST /api/shipments/{shipment:reference}/cancel: Gate::authorize('cancel', *shipment*) ثم *cancel*(*shipment* : Shipment) : داخل معاملة، أعِد قراءة الشحنة باستخدام *lockForUpdate*() واستدعِ *findOrFail*(shipment->id) : BookingService() : transitionTo(ShipmentStatus::Cancelled, 'Cancelled by customer.'). الدالة transitionTo تُرجع أصلاً 422 مع errors.status عندما يكون الانتقال غير مسموح. أرجِع 200 مع ShipmentResource.
3. GET /api/shipments/{shipment:reference}/tracking: Gate::authorize('view')، ويُرجع ShipmentEventResource::collection للأحداث مرتبةً تصاعدياً حسب occurred_at: status، note، occurred_at. الاختبارات: إلغاء شحنة confirmed ← 200 ووجود حدث؛ وإلغاء in_transit ← 422 والحالة لم تتغير؛ وإلغاء شحنة ملغاة أصلاً ← 422؛ وعميل آخر ← 403؛ والتتبّع يُرجع الأحداث بالترتيب؛ وتتبّع شحنة عميل آخر ← 403؛ وإلغاء نسخة قديمة يعيد التحقق من أحدث حالة. استخدم الحالة status() في ShipmentFactory.

بالإنجليزية:

```
Plan first, then wait for my approval. Tests first, small diffs.

1. Add cancel(User, Shipment) to ShipmentPolicy: the owner or staff.
2. POST /api/shipments/{shipment:reference}/cancel:
   Gate::authorize('cancel', $shipment), then BookingService::cancel():
   in a transaction, re-read the shipment with
   Shipment::lockForUpdate()->findOrFail($shipment->id) and call
   transitionTo(ShipmentStatus::Cancelled, 'Cancelled by customer.').
   transitionTo already returns 422 with errors.status when the move is
   not allowed. Return 200 with ShipmentResource.
3. GET /api/shipments/{shipment:reference}/tracking: Gate::authorize('view'),
   returns ShipmentEventResource::collection of events ordered by
   occurred_at ascending: status, note, occurred_at.
Tests: cancel confirmed -> 200 and an event exists; cancel in_transit -> 422
and status unchanged; cancel already cancelled -> 422; another customer
-> 403; tracking returns events in order; tracking for another customer's
shipment -> 403; cancelling a stale copy re-checks the latest status.
Use ShipmentFactory's status() state.
```

ما الذي تتحقق منه.

- جيد: يُتحقق من قاعدة الحالة عبر `transitionTo()` / `canTransitionTo()`؛ ولا توجد قائمة بالحالات في المتحكم.

- جيد: يتحقق اختبار الشحنة قيد النقل من أن الحالة *ما زالت* `in_transit` في قاعدة البيانات بعد 422 (`$shipment->refresh()`).
- علامة خطر: يُرمى خطأ 422 بصيغة `abort(422, ...)`. هذا يعطي `{"message": ...}` دون `errors`، مما يكسر جدول الأعراف. استخدم `ValidationException::withMessages`.
- علامة خطر: يحدث فحص السياسة وفحص 422 بالترتيب الخاطئ. يجب أن يحصل الغريب على 403، لا 422 (وإلا عرف حالة الشحنة).

```
curl -s -X POST http://localhost/api/shipments/$REF/cancel \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" | jq '.data.status'
```

```
"cancelled"
```

```
curl -s http://localhost/api/shipments/$REF/tracking \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN"
```

```
{
  "data": [
    {"status": "pending", "note": "Booking received.", "occurred_at": "2026-10-
        21T11:40:02.000000Z"},
    {"status": "confirmed", "note": "Booking confirmed.", "occurred_at": "2026-10-
        21T11:40:02.000000Z"},
    {"status": "cancelled", "note": "Cancelled by customer.", "occurred_at": "2026-10-
        21T14:58:40.000000Z"}
  ]
}
```

ألغِها مرة أخرى:

```
{"message":"A cancelled shipment cannot move to cancelled.","errors":{"status":["A cancelled
        shipment cannot move to cancelled."]}}
```

الخطوة 8 – قائمة تحقق مراجعة الشيفرة بالذكاء الاصطناعي (15 دقيقة)

ماذا ولماذا. قبلتَ اليوم كثيراً من شيفرة الوكيل. والآن راجعها كما لو أن زميلاً فتح طلب سحب (Pull Request). أولاً **يدوياً**، ثم مع الوكيل، ثم قارن.

> **تحذير**
>
> **يدوياً أولاً.** اقضِ 8 دقائق مع `git diff day1-end --stat`، ثم اقرأ المتحكمات والطلبات والسياسة. املأ قائمة التحقق أدناه على الورق قبل أن توجّه الطلب إلى الوكيل.

#	الفحص	أين تبحث
1	كل مدخل يُتحقق منه في FormRequest؛ وتستخدم المتحكمات `validated()`	`app/Http/Requests`، المتحكمات
2	لا إسناد جماعي لـ `role` أو `user_id` أو `price_lyd` أو `status` من المدخلات	`$fillable`، الطلبات
3	كل مسار باستثناء التسجيل وتسجيل الدخول والمدن محمي بـ `auth:sanctum`	`sail artisan route:list --path=api`
4	كل قراءة أو كتابة لشحنة تمر بتفويض السياسة	المتحكمات
5	المال من النوع `decimal`، ويُسلسَل سلسلةً نصية	الترحيلات، الموارد
6	الكتابات على عدة صفوف داخل `DB::transaction`	`app/Services/BookingService.php`
7	الأخطاء تتبع جدول الأعراف (422 فيها `errors`)	الاختبارات
8	الاختبارات تغطي المسارات غير السعيدة، لا السعيد وحده	`tests/Feature`
9	لم يُضعَف أي اختبار لتنجح الشيفرة	`git log -p tests/`
10	لم يُضَف شيء لم تطلبه المواصفة	الفرق كاملاً

```
sail artisan route:list --path=api
```

```
  POST       api/auth/login ......................... Api\AuthController@login
  POST       api/auth/register ...................... Api\AuthController@register
  GET|HEAD   api/cities ............................. Api\CityController@index
  POST       api/quotes ............................. Api\QuoteController@store
  GET|HEAD   api/shipments ...................... Api\ShipmentController@index
  POST       api/shipments ...................... Api\ShipmentController@store
  GET|HEAD   api/shipments/{shipment} ............ Api\ShipmentController@show
  POST       api/shipments/{shipment}/cancel ... Api\ShipmentController@cancel
  GET|HEAD   api/shipments/{shipment}/tracking Api\ShipmentController@tracking
```

ثم أجرِ المراجعة نفسها عبر الوكيل:

🤖 **اسأل وكيلك الذكي**

طلب للوكيل

لا تغيّر أي ملفات. راجع git diff day1-end بصفتك مراجعاً خبيراً مستخدماً قائمة التحقق هذه: مدخلات متحقَّق منها فقط؛ ولا إسناد جماعي لـrole أو user_id أو price_lyd أو status؛ auth:sanctumg على كل مسار باستثناء register loging وcitiesg؛ وسياسة على كل قراءة وكتابة للشحنات؛ والمال بنوع decimal وسلسلةً نصية؛ ومعاملات حول الكتابات على عدة صفوف؛ وشكل الأخطاء يطابق الافتراضيات في Laravel؛ ووجود اختبارات للمسارات غير السعيدة؛ وعدم إضعاف أي اختبار؛ وعدم وجود ميزات غير مطلوبة. أجب عن كل بند بـPASS أو FAIL مع دليل بصيغة file:line.

بالإنجليزية:

```
Do not change any files. Review `git diff day1-end` as a senior reviewer
using this checklist: validated input only; no mass assignment of role,
user_id, price_lyd or status; auth:sanctum on every route except register, login and
cities;
policy on every shipment read and write; money as decimal and string;
transactions around multi-row writes; error shape matches Laravel defaults;
unhappy-path tests exist; no test was weakened; no unrequested features.
For each item answer PASS or FAIL with file:line evidence.
```

قارن إجابات الوكيل بإجاباتك. وحيثما اختلفتما، افتح الملف وقرّر من المحق. أصلح حالات FAIL الحقيقية في إيداعات صغيرة ومنفصلة.

نقطة تحقق

نقطة التحقق 2 (15:45)

```
sail artisan migrate:fresh --seed
sail artisan test
```

```
   PASS  Tests\Unit\PricingServiceTest
   PASS  Tests\Unit\ShipmentStatusTest
   PASS  Tests\Feature\AuthTest
   PASS  Tests\Feature\CityTest
   PASS  Tests\Feature\QuoteTest
   PASS  Tests\Feature\ShipmentPolicyTest
   PASS  Tests\Feature\ShipmentTest
   ...
  Tests:    56 passed
```

يحتوي المرجع `day2-end` على **56** اختباراً (23 من اليوم الأول، و33 جديداً). تحتاج إلى **20 اختباراً جديداً على الأقل اليوم** وصفر إخفاقات؛ وسيختلف مجموعك.

اختبار دخان شامل (End-to-End Smoke Test) (قاعدة بيانات جديدة، لذا سجّل مرة أخرى):

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/api/shipments -H "Accept:
        application/json"
# 401
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/api/shipments -H "Accept:
        application/json" -H "Authorization: Bearer $TOKEN"
# 200
```

```
git add -A
git commit -m "feat: shipments, policy, cancel and tracking"
```

إن واجهتك صعوبة

الحل المرجعي هو الوسم (Tag) `day2-end`.

```
# Which files differ from the reference?
git diff --stat day2-end

# Read the reference version of a file without switching branches
git show day2-end:app/Policies/ShipmentPolicy.php

# Diff one file against the reference
git diff day2-end -- app/Http/Controllers/Api/ShipmentController.php

# Run the reference tests against your code (copy one test file over)
git show day2-end:tests/Feature/ShipmentTest.php > tests/Feature/ShipmentRefTest.php
sail artisan test --filter=ShipmentRef
```

هذه الحيلة الأخيرة قوية: فاختبارات المرجع مواصفة مجانية. إن فشلت على شيفرتك، فاقرأ السبب.

لتبدأ من جديد انطلاقاً من المرجع (يبقى عملك على `my-day2`):

```
git stash
git switch -c my-day2-reset day2-end
sail artisan migrate:fresh --seed
sail artisan test
```

أهداف إضافية للتحدي

1. **قيود الفحص.** أضف قيود فحص (Check Constraints) في PostgreSQL هي `CHECK (price_lyd ⩾ 0)` و`CHECK (weight_kg > 0)` على `quotes` و`shipments`، واختباراً يثبت أن قاعدة البيانات ترفض الصفوف الخاطئة حتى عند تجاوز التطبيق.
2. **المال بالأعداد الصحيحة.** غيّر `PricingService` ليحسب بالدراهم أعداداً صحيحة (أو استخدم `bcmath`) بدلاً من الأعداد العشرية العائمة، مع إبقاء كل اختبارات الخطوة 1 ناجحة. اشرح في جملة واحدة لماذا تُعدّ الأعداد العشرية العائمة خطرة على المال.
3. **تسجيل الخروج.** يحذف `POST /api/auth/logout` الرمز الحالي فقط (`$request→user()→currentAccessToken()→delete()`). اختبر أن الرمز القديم يُرجع بعدها 401.
4. **التصفية.** `GET /api/shipments?status=pending` مع التحقق باستخدام `Rule::enum` على سلسلة الاستعلام (Query String). اختبر أن الحالة غير الصالحة تعطي 422.
5. **قائمة الموظفين.** يرى المرسِلون والمديرون جميع الشحنات في `GET /api/shipments`. قرّر هل ينبغي أن تكون هذه نقطة النهاية نفسها أم مساراً جديداً `/dispatch/shipments`، وبرّر قرارك.

التأمّل

1. أي اختبار حاول الوكيل تغييره أو تخطيه أو إضعافه اليوم، إن وُجد؟ وإن لم يوجد، فأي اختبار قرأته بأكبر قدر من العناية، ولماذا؟
2. جد موضعاً واحداً كانت فيه شيفرة الوكيل صحيحة لكنها لم تتبع جدول أعراف الاستجابة (رمز الحالة، أو شكل الخطأ، أو المال سلسلةً نصية). كيف وجدته: باختبار، أم بـcurl، أم بالقراءة؟
3. في الخطوة 8، أين اختلفت مراجعتك اليدوية عن مراجعة الوكيل؟ ومن كان المحق؟